

BACHELOR INFORMATICA



UNIVERSITY OF AMSTERDAM

Effective Geospatial Data Discovery and Extraction

Vigo Vlugt

June 23, 2023

Supervisors: Gabriel Pelouze, Zhiming Zhao

Abstract

This research paper proposes an application that allows researchers to efficiently browse, select, and extract geospatial data from data catalogues via a Jupyter environment. The application addresses gaps in existing work by supporting a wide range of public geospatial metadata catalogues, integrating well with Jupyter, and streamlining the research pipeline. Through usability testing, the application demonstrates its effectiveness in browsing catalogues, reviewing metadata records, previewing data, dataset subsetting, and exporting datasets to Jupyter. Overall, this research contributes to the field by providing a tool that enhances the efficiency of working with geospatial data, enabling researchers to make informed decisions, accelerate their research, and improve decision-making processes in various fields.

Contents

1	Introduction	7
2	State of the art & related work	9
2.1	Theoretical background	9
2.1.1	Geospatial data types	10
2.1.2	Geospatial visualization	10
2.2	Related technologies	10
2.2.1	Geospatial data standards	10
2.2.2	Geospatial data servers	11
2.2.3	Metadata catalogue protocols	12
2.2.4	Metadata catalogues	12
2.2.5	Metadata record standards	13
2.2.6	Map viewers	14
2.2.7	OGC API	14
2.3	Related work	15
2.3.1	Geospatial data and Jupyter	15
2.3.2	Subsetting systems	16
2.3.3	Geospatial metadata catalogues	16
2.3.4	Gap identification	17
3	Application design	19
3.1	User Stories & Requirements	19
3.2	System architecture	20
4	Implementation	23
4.1	Technical choices	23
4.1.1	Map viewer	23
4.1.2	Metadata catalogue protocols	24
4.1.3	Metadata standards	25
4.1.4	Client libraries	25
4.1.5	Cross-Origin Resource Sharing	25
4.1.6	Jupyter integration	27
4.2	Technical Architecture	27
4.3	Current status	30
5	Demonstration	31
5.1	Selecting an CSW endpoint	31
5.2	Browsing the metadata catalogue	32
5.3	Service view	33
5.4	Exporting data	33
5.4.1	Exporting a downloadable resource	34
5.4.2	Exporting a feature type	34
5.4.3	Exporting a subset of features	35

6	Experimental results	39
6.1	Results	40
7	Discussion	43
7.1	Usability & impression test	43
7.2	CSW & metadata catalogue usage	44
7.3	Ethical considerations	45
8	Conclusion	47
A	Appendix	53
A.1	Source code	53
A.2	Usability form	53

Introduction

Large geospatial datasets are becoming increasingly available to the public. As a result, this development opens up new opportunities for research and innovation in a wide variety of fields, from wildlife preservation to epidemiology. Public access to these datasets enables researchers to make well-informed decisions that profoundly affect the world around us.

While the massive amount of available datasets provides a lot of opportunities, this also means that researchers must consider a lot of datasets to select the best one for a project. Moreover, there are often also multiple different dataset sources that have to be compared. Evaluating a dataset can be challenging as there are many factors to consider [45]. Especially for geospatial data, it is essential to visualise and explore the dataset to gain insight into the spatial distribution and context of the data. This will also improve the understanding of the dataset, accelerating the research process. Ultimately, the choice of the dataset can significantly impact the outcome of a research project.

Often, these datasets are of a considerable size due to the exponential growth of data in recent years [43]. However, analysis of this data does not always require the entire dataset; sometimes, only a subset of the complete dataset is relevant to the research. Even if all the data is relevant, analysing a subset could make the research process easier, quicker and cheaper. For geospatial data, a flexible method of creating a subset would be to let the researcher select the relevant points visualised on a map. By visually exploring the data, patterns and trends can be identified, leading to better-informed decisions about which subset of the data is most relevant for analysis. This process of visually selecting a subset is not integrated into current metadata catalogue tools, as most tools only allow for exporting the complete dataset.

After selecting a dataset, researchers will begin their analysis. In recent years, the scientific computing environment Jupyter has become a popular tool for researchers and developers [41] to use for research projects. Jupyter allows for creating documents (Jupyter Notebooks) that contain live code, output, visualisations, equations and explanatory text. This code can be written in a multitude of different languages, such as R, Julia or Python, which is currently the most popular language in data science. The Jupyter environment can be accessed in the browser and connects to a backend that can be run on a local- or remote computer, making Jupyter very accessible. Moreover, researchers can collaborate on Jupyter Notebooks in real-time and easily share these notebooks to facilitate knowledge sharing. For these reasons, Jupyter is a great fit for geospatial data analysis.

When working with geospatial data in Jupyter, the geospatial data must first be extracted from the catalogue and then imported into Jupyter. While data catalogues often provide a way to download the data, there is no dedicated integration with Jupyter.

In the field of data science, there is a pressing need for efficient methods to help geospatial data researchers browse catalogues, select a dataset and import this data into Jupyter. As

indicated, these tasks are time-consuming, which is why it is crucial to enable researchers to perform this process as efficiently as possible. By addressing these needs, this research aims to streamline the data exploration, selection, and extraction process, enabling data scientists to make well-informed decisions and accelerate their research workflow.

This thesis will address the research question: How to enable data science researchers to effectively browse, select and extract geospatial data from data catalogues via a Jupyter environment? To answer this question, we define the following sub-questions:

- What are the current practices of browsing, selecting, and extracting geospatial data via a Jupyter environment?
- What are the common formats and protocols for accessing geospatial data?
- How to design and implement a tool which provides a solution to the research question.
- How can you effectively export the selected data into Jupyter such that it can be easily analyzed?

In this thesis, we will make the following contributions:

- A literature review of the related work and theoretical background.
- An architectural schema of the tool.
- A prototype of the tool based on the architectural schema.

This prototype will allow users to visualize spatial boundaries of datasets in geospatial data catalogues in a web-based environment. Furthermore, the user can select a dataset to review its metadata, and if the dataset supports it, show the data in the browser. The user will then be able to select the relevant data and easily export it to a Jupyter Notebook.

The proposed prototype is significant as it addresses the growing need for effective browsing, selecting and extracting geospatial data from data catalogues and repositories. With the increasing availability of large public geospatial datasets, it is essential that researchers can effectively make an informed decision on what dataset to use for a project. The prototype has the potential to accelerate the research process and could help researchers to select better datasets, which could, in turn, lead to more accurate and insightful analysis, and, ultimately, better decision-making.

State of the art & related work

In this chapter, we will first review the necessary theoretical background for the field of geospatial data. Next, we will discuss several relevant technologies that will be considered to be integrated into the prototype. These technologies will aid in answering the sub-research question: What are the common formats and protocols for accessing geospatial data? Finally, we will examine current relevant work that will inform the design and implementation of the prototype. This will include a review of existing tools to work with geospatial data, the workflow of working with geospatial data in Jupyter, as well as recent research and trends in the field. By understanding the theoretical foundation, relevant technologies, and relevant work of geospatial data, we can develop a prototype that will effectively answer the research question.

2.1 Theoretical background

Geospatial data is a type of data associated with a location on Earth and is often referred to as geographic or spatial data. While geospatial data can also refer to three-dimensional data, it often refers to two-dimensional data. This data can include information about the physical features of the Earth's surface, such as terrain type or elevation, as well as human-created features, such as buildings, roads, and regional boundaries.

Geographic Information Systems (GIS) are systems that store, analyze and visualize geospatial data [6]. These systems are used in a wide variety of domains, including governing, natural resource management, and business, and are integral in geospatial data analysis. Besides storing data, GIS systems are also often used for data visualization, allowing researchers to create visual representations of geospatial data in the form of a map. Overall, GIS systems are an essential tool for many fields and industries, as they provide a powerful system for managing and analyzing geospatial data.

Geospatial metadata catalogues, also known as geospatial data catalogues, allow organizations to share geospatial datasets with other organizations and researchers. These catalogues can be used to discover and access public datasets. An example of an established catalogue is the European Environment Agency (EEA) geospatial catalogue [44], which provides and maintains more than 2000 geospatial datasets covering topics from agriculture and food. The datasets' data are usually not stored in the catalogue but on different geospatial data servers. Rather, catalogues store and provide metadata records which contain information about the datasets. This information includes a description, the geographic extent, the temporal extent, the license and a reference to where the data is stored. In a later section, we will review the most common formats for this metadata, as these metadata records are typically provided in a standard format.

2.1.1 Geospatial data types

There are two main types of geospatial data, vector-based and raster based. Raster-based data is structured in a grid where each cell is associated with a geospatial location and contains a data value that corresponds to that location [51]. Raster data is commonly used to represent continuous spatial data such as temperature or elevation. Besides continuous data, raster data is also capable of representing discrete data, such as the type of land. Another type of raster data is basic imagery that does not represent anything other than pixels, often from satellites or aerial photography.

Vector data represents geometric features using geometric shapes such as points, lines, and polygons [7]. These geometric shapes define the position or extent of the feature. For example, a point could indicate the centre of a city, lines are often linked together and can represent roads, and polygons could indicate the boundaries of a municipality. Besides the shape, the feature contains attributes associated with that feature, such as the city's name or number of inhabitants.

2.1.2 Geospatial visualization

Geospatial visualization is the process of visualizing geospatial data. As the data has a spatial component, it often makes sense to visualize it on a map. Map visualization can help users interpret patterns in the data and find relations with the environment that otherwise would be difficult to find. Since the release of HTML5, map visualization has become much more accessible to users, as users do not have to download large applications.

2.2 Related technologies

The Open Geospatial Consortium (OGC) [35] is an organization which has had a significant impact on geospatial technologies since its founding in 1994. The organization's main objective is to develop publicly available open standards for geospatial data and technologies. The goal is to improve and facilitate the exchange of geospatial data between different organizations through the interoperability of various standards. To achieve this goal, the OGC makes decisions using a member-driven consensus process to avoid the control of any single organization or company. The standards created by OGC have been widely adopted within the industry and are used by thousands of organizations internationally [33].

Another organization which has had a big impact on geospatial technologies is the Open Source Geospatial Foundation (OSGeo) [2]. OSGeo is a non-profit organization which develops open-source geospatial software applications and libraries. One of their missions is to foster the global adoption of geospatial technology and to promote interoperability with existing standards.

2.2.1 Geospatial data standards

One such standard is the Web Map Service (WMS) [37], which is created by the OGC. This standard defines an HTTP interface which enables requesting georeferenced map images. Typically, these images are formatted as PNG, JPEG or GIF; therefore, the standard WMS protocol only supports raster data. The defining property of this standard is that every image returned is dynamically rendered, which allows for flexibility regarding the map data and styling used to create the image. The WMS standard specifies the number of operations that the user can request from the service, of which the following are the most important:

- GetCapabilities - is a discovery operation that returns WMS metadata, such as the bounding box and layers of data.
- GetMap - is the main operation of any WMS; it returns an image based on the layer, bounding box, and styles provided by the client in the request.

- GetFeatureInfo - is an operation which returns metadata for a specified coordinate on the map.

The Web Map Service standard has become an essential part of geospatial data distribution. However, the dynamic rendering nature of the protocol can result in slow request times and high computational load on the server.

To address these issues, the Web Map Tile Service (WMTS) standard was developed [37]. Unlike WMS, WMTS prerenders the map before serving the data to clients. When a client then requests a part of the map, a cached image is served, reducing the load on the server and speeding up the request times significantly. However, this also means that a client cannot specify a custom bounding box, as the images are rendered in a predefined grid with predefined zoom levels. Moreover, the client can not specify a custom style for these images, as the style must be chosen before the map is prerendered. Finally, the prerendering of the entire map is a process which typically takes a lot of time. The operations supported by WMTS are very similar to the WMS operations.

- GetCapabilities - is a discovery operation which returns metadata about the dataset.
- GetTile - returns a prerendered map tile based on the predefined zoom level and tile coordinates provided by the client in the request.
- GetFeatureInfo - is an operation which returns metadata for any coordinate on the map tile, as the coordinate can be anything specified by the client; this is a dynamic request and must be processed on the server.

Overall, the Web Map Tile Service standard offers faster and more efficient access to geospatial data but is limited in regards to customization.

WMS and WMTS only support raster data; therefore, OGC developed the Web Feature Service (WFS) designed to serve vector data. Instead of images, a WFS serves map data in various vector data formats. The default format is Geography Markup Language (GML) which all compliant WFS must support. Other formats, such as the popular GeoJSON [8] or Shapefile format [13] can also be supported, but the specification does not require this to be WFS compliant. The client can specify which properties of the features to fetch when requesting data. Additionally, the client can use advanced filters based on spatial and non-spatial properties of the features to ensure that only relevant data is returned. The operations WFS supports are also structured in the same way as WMS and WMTS.

- GetCapabilities - returns metadata about the WFS, such as the different types of features and the capabilities of the supported filters.
- GetFeature - is the operation corresponding to GetTile and GetMap; in the request, the client can send the bounding box, the filters and the desired output format. The WFS then responds with the corresponding data.
- DescribeFeatureType - returns metadata about one type of feature, this metadata contains the schema of the data and the types of geometry features of this type can have.

For vector data, WFS has become the de facto standard for serving and accessing geospatial data over the web.

2.2.2 Geospatial data servers

These standards are implemented by geospatial data servers, which store geospatial data and expose it to users. The open-source project GeoServer [16] is among the most widely used geospatial data servers [21]. It allows users and organizations to share and publish geospatial data in vector and raster formats. Many protocols developed by OGC are implemented, including WFS, WMS and WMTS; therefore, GeoServer is very interoperable with other services. Furthermore,

a web mapping solution is built into GeoServer so users can easily view and edit their maps.

Although less popular than GeoServer, MapServer [31] is another widespread open-source geospatial data server. While GeoServer is written in Java, MapServer is written in the more low-level language C, which is why it is considered the faster server of the two. MapServer also has extensive support for OGC protocols; most notably, MapServer implements the WFS, WMS and WMTS standards.

2.2.3 Metadata catalogue protocols

While WFS, WMS and WMTS are used for exchanging geospatial data with geospatial servers. The purpose of metadata catalogue protocols is to enable programmatic access to the metadata records in a metadata catalogue.

Catalogue Service for the Web (CSW) [36] is a metadata catalogue protocol created by the OGC. CSW supports a rich query language, which allows clients to filter metadata record results based on keywords and spatial and temporal properties. The operations supported by CSW are comparable to those supported by WFS, WMS and WMTS:

- GetCapabilities - this operation returns information about the CSW server.
- GetRecords - this operation allows clients to search for metadata records using the CSW query language.
- GetRecordById - this operation returns the full metadata for a specific record.

The metadata records stored in CSW are often also in a standardised format. CSW is widely used and supported in many metadata catalogues.

There are several often-used metadata protocols other than CSW. These are GeoNetwork API, OAI-PMH and OpenSearch.

The GeoNetwork API [18] is a protocol specifically for GeoNetwork metadata catalogues which are one of the most used metadata catalogues. This API also exposes endpoints to search and list metadata records in the GeoNetwork server. GeoNetwork API is, however, only supported by GeoNetwork and thus not a standardised protocol.

OpenSearch [39] is a protocol commonly used to provide search functionality with a standardised format of queries and search results. OpenSearch is not specific to geospatial data and can be used for any web service which wishes to offer standardised search functionality. The results of OpenSearch are often serialised in the Atom [48] format, which is an XML-based standard for publishing information feeds and is thus commonly used for blogs or RSS feeds. INSPIRE Atom [47] is an extension of Atom created by the European Union which builds upon Atom by adding several geospatial-related attributes. In the INSPIRE technical guidelines, an architecture which combines OpenSearch and INSPIRE Atom is defined to facilitate search functionality for metadata records. While this service is natively implemented in GeoNetwork, pycsw only supports standard OpenSearch.

The final metadata record exchange protocol to be discussed is the Open Archives Initiative Protocol for Metadata Harvesting (OAI-PMH) [49]. OAI-PMH is a protocol that enables the harvesting of metadata records from dataset repositories. The protocol is often used in academic settings and is not specific to geospatial metadata. Both GeoNetwork and pycsw implement this protocol by default.

2.2.4 Metadata catalogues

GeoNetwork [15] is a metadata catalogue for managing geospatial data developed by the same organization that created GeoServer, OGC. Similar to GeoServer, it is also free, open-source and widely adopted in the industry. For researchers, the main functionality of GeoNetwork is the sharing and discovery of geospatial datasets. GeoNetwork provides a web-based interface that allows researchers to search and browse these datasets. In the web interface, researchers can also add

filters to search for datasets based on keywords, temporal or spatial extent, data types and data formats. Moreover, a web mapping library is also included so these datasets can be interactively visualised in the web interface. For administrators, the main functionality of GeoNetwork is its metadata record managing capabilities of local and distributed servers. GeoNetwork supports a wide range of geospatial metadata protocols, under which: CSW, GeoNetwork API, OAI-PMH and OpenSearch. Furthermore, GeoNetwork supports various metadata record formats, which allows for integration with other systems.

Other prevalent metadata catalogue servers include pycsw, GeoNode and CKAN . Pycsw [42] is primarily a CSW implementation written in Python; it can be used as a library or deployed with a web server. Despite what the name suggests, pycsw implements other widely used metadata protocols besides CSW, such as OpenSearch and OAI-PMH. Pycsw does not have a graphical user interface out of the box. GeoNode [50] is both a geospatial data server and a geospatial metadata server. GeoNode uses pycsw internally and adds a user-friendly graphical user interface on top of this. As a consequence, GeoNode supports the same metadata protocols as pycsw. CKAN is a data management server not specifically tailored to geospatial data. It is widely used by governments to make their data accessible to the public. Although CKAN is not designed for storing geospatial data, the 'ckanext-spatial' extension extends CKAN's capabilities by providing features specific to geospatial data, such as CSW support. One such new feature is geospatial metadata API support which is also powered by pycsw.

2.2.5 Metadata record standards

The metadata records stored in metadata servers and exchanged using metadata protocols have a standardised format. In this sub-section, the most common geospatial metadata record standards will be reviewed.

ISO 19115 [24] is a metadata record standard created by the International Organization for Standardization (ISO), primarily used for describing datasets that have geographic extents. The standard is widely used in the geospatial community, GeoNetwork and pycsw have built-in support for ISO 19115, and GeoNetwork also provides an integrated ISO 19115 metadata editor. The standard aims to solve the following problems:

- To allow dataset managers to describe and characterise their datasets.
- To assist researchers with dataset discovery.
- To disclose the information, researchers need to decide whether or not a dataset is relevant to them.
- To inform researchers about how to access the dataset.

ISO 19115 defines the metadata's content and structure but not the content's encoding. Encoding of ISO 19115 is defined in another standard, namely ISO 19139 [25]. This allows for the exchange of ISO 19115 metadata between different systems and facilitates interoperability between other organisations. The ISO 19139 format is based on the eXtensible Markup Language (XML) format, which many programming languages have built-in parsing and serialising support for.

Besides ISO 19115 and ISO 19139, other widely used metadata formats exist. One example is Dublin Core [12], which is a metadata standard that is not only used for geospatial resources but created to be a solution for a wide range of domains. It is natively integrated into GeoNetwork and pycsw, and it is the default metadata format of the OAI-PMH protocol. Other common metadata formats are INSPIRE Metadata [22] and CSDGM [10], created by the European Union and the US Federal Geographic Data Committee (FGDC), respectively. The INSPIRE metadata format extends ISO 19115 by providing additional elements to accurately and fully describe the

data. Content Standard for Digital Geospatial Metadata (CSDGM) is an older format many organizations have used; however, the FGDC currently advocates for projects to use an ISO metadata standard.

2.2.6 Map viewers

As this research is oriented around Jupyter, which is web-based, we will only look at web-based geospatial visualization. Currently, the most popular mapping solutions for the web are Leaflet, OpenLayers, Mapbox and Maplibre.

Of these libraries, Leaflet [28] is the most lightweight solution; it is designed to be easy to use and flexible. Leaflet is one of the most well-known open-source web mapping libraries and is used in many projects. Developers can create layers based on geospatial data, which are then placed on top of each other on the map. Out of the box, Leaflet supports WMS, WMTS, GeoJSON, Vector, and Tile layers. A rich plugin system allows support for other types of layers, such as WFS, which are not natively supported. A drawback of Leaflet is that only the EPSG:3857 projection is supported; however, plugins can solve this problem.

OpenLayers [38] is the oldest library of these solutions, but very powerful and feature-rich. It is created by the Open Source Geospatial Foundation, the same foundation behind GeoNetwork and GeoServer. OpenLayers has built-in support for many more advanced features, such as WFS layers and other projections; therefore, third-party plugins are unnecessary in most situations. Just like Leaflet, OpenLayers is open-source and successful in its adoption. Compared to Leaflet, OpenLayers is the web mapping solution that is built-in into more advanced GIS systems, while Leaflet is used more for basic web mapping. GIS systems which use OpenLayers as their map viewer library include GeoNetwork and GeoServer.

Mapbox [29] is a commercial platform that offers a multitude of services regarding geospatial visualization and analysis. The most interesting service Mapbox offers for this project is its web mapping library Mapbox GL JS. While version 1 of this library was open-source, in 2020, Mapbox introduced version 2, which switched to a proprietary license. Consequently, version 2 is only usable with Mapbox products, which can be expensive. Both version 1 and 2 of Mapbox are implemented with WebGL, which results in high-performance rendering.

Mapbox version 1 introduced the Mapbox Vector Tile (MVT) specification, which encodes vector geospatial data in a compact binary format based on Google protocol buffers. This specification has become popular due to its compact format and efficient rendering capabilities. As such, OpenLayers has native capabilities for rendering MVT data, and GeoServer recommends using MVT format for rendering vector tiles.

To continue the development of the open-source version of Mapbox, the open-source community created a fork called MapLibre [30]. MapLibre has the same API and functionality as MapBox version 1 while adding new features and maintaining the codebase.

Mapbox and MapLibre are both very good at rendering vector maps from the MVT format. Other functionality, such as WMS rendering is possible, but both libraries are not as feature-rich as OpenLayers as they miss important functionality, such as WFS support.

2.2.7 OGC API

In the previous sections, we have seen many standards designed by the OGC: WMS, WFS, WMTS and CWS. These standards were published in 1999, 2002, 2010 and 2004, respectively, and while they are all still very widely used, they are also dated. Firstly, the default serialization format for these standards is XML, which has a declining usage. Secondly, the request/response formats are somewhat complex and verbose. To address these issues, the OGC is currently developing a new generation of these standards, the OGC APIs [34].

OGC APIs are designed to use more modern technologies and be more efficient and flexible. The new generation of APIs will use modern web development practices, such as representational state transfer (REST) and the JavaScript Object Notation (JSON) format for serialization. Another

significant change is that the new APIs will also offer improved support for Linked Data, which improves integration with other data sources.

The new OGC APIs are still in the draft stage and have thus not yet seen widespread use in the ecosystem. Nevertheless, these OGC APIs may be expected to replace the old OGC standards and become the new default in the coming years.

2.3 Related work

In this section, we review and analyze the existing body of work that is related to this study. The selected related work has been determined by searching for studies which are closely related to the research question: How to enable data science researchers to effectively browse, select and extract geospatial data from data catalogues via a Jupyter environment? Based on the discovered related work, three general categories have been identified. Namely: Case studies which integrate geospatial data with Jupyter, case studies which allow for the subsetting of datasets, and lastly, other geospatial metadata catalogue systems.

2.3.1 Geospatial data and Jupyter

In recent years Jupyter Notebooks have emerged as a popular environment for data exploration, processing, and visualization. We will now go over some papers which integrate geospatial data with Jupyter, either providing data processing, discovery, visualization or analysis. By reviewing papers which explore this integration, we hope to find an answer to the sub-research question: What are the current practices of browsing, selecting, and extracting geospatial data via a Jupyter environment?

In [46] a computing platform is proposed which can store Petabytes of data on a Distributed File System (DFS). The platform also includes an integrated Jupyter Notebook environment for data discovery, processing, and visualization. However, the system is designed to work on the data stored on their DFS, not with other catalogues. Furthermore, the system's main functionality enables users to submit large-scale batch-processing jobs directly from the notebook. The data discovery and selection are not the main focus of this project, nor is the analysis directly in the Jupyter Notebook.

An application for interactively visualising geospatial data is developed in [3]. The application can visualise meteorological data and supports time-series data; interaction with this data is also possible. Furthermore, the application is integrated with Jupyter and provides a Python interface to import the data. Unfortunately, the application is not developed for selecting and exporting data, and the application solely supports meteorological data.

[3] presents two applications developed using Jupyter Notebook. These applications are embedded in Jupyter and allow for interactive querying, analysis and visualisation of geospatial data. In the second chapter, an overview of solutions for various geospatial problems in a Jupyter and Python context is shown, including the packages Ipywidgets and Folium.

Ipywidgets (or alternatively Jupyter Widgets) [27] is a Python package which allows for interactive controls in Jupyter notebooks. Users can then create interactive cells with elements such as sliders or text inputs that are bound to a variable in Python. Interacting with these elements then rerenders the cells that depend on these variables.

Folium [14] is a package for visualising data in Jupyter on an interactive map inside a cell in the notebook. On the map, you can zoom, pan around, and visualise any geospatial data stored in the notebook.

The first application developed seeks to apply spatial visualisation to analyse the supply and demand of teachers and vacancies in higher education courses. The data used in this analysis were stored in a proprietary PostGIS database. This data was then visualised on a Folium map, while the user could interactively filter the map using a Jupyter Widget. The second application described is similar to the former application, as the architecture is the same. This application visualises emotions associated with an environment, seeking to apply this information in urban planning.

This study and its associated applications offer a comprehensive overview of interactive geospatial analysis solutions utilising Jupyter. However, they do not address the issue of exploring and comparing diverse datasets before initiating an in-depth analysis. Nonetheless, the libraries suggested in this study have the potential to be utilised for this purpose.

2.3.2 Subsetting systems

In this subsection, we will go over studies which explore various methods of subsetting data in order to extract a part of the dataset which is relevant to the researcher. Doing this will aid us in designing the selection functionality of the application.

In [20], an algorithm is proposed to efficiently select and explore a representative subset of objects in an area that the user is interested in. The main requirements of this study are that the subset must contain the most representative points and that these points must not visibly cover other points when visualized on the map. While the study defines great strategies to visualise points on a map, it does not provide a method to extract this data to a Jupyter environment easily. Furthermore, the emphasis of the study is on selecting a subset of the data, which is not necessarily what we want to do for this project. However, the algorithm could be useful when we develop the visualisation of a large number of points.

[19] describes a system that allows users to request geospatial data through a web interface. The requests are processed on a High-Performance Computing (HPC) cluster, which then produces a Comma Separated Values (CSV) file which contains the requested data. While the platform includes a web interface, the system does not allow for fine-grained selection. Only boundaries of countries can be specified to select data. Users can import any dataset, but there is no visualisation for this dataset, and no interaction with the individual data points is possible. Moreover, the main export method integrated is as a CSV file, which may not be the optimal format for working with geospatial data in Jupyter.

2.3.3 Geospatial metadata catalogues

For this subsection, we will not review studies, but an application which is particularly relevant to the research question, namely the open-source metadata catalogue application, GeoNetwork. This will inform us more generally about how we could design our application as a whole.

As discussed in the previous chapter, GeoNetwork [15] is one of the most used geospatial metadata catalogue applications. Besides storing metadata records, GeoNetwork provides an advanced user interface where users can search and browse these metadata records.

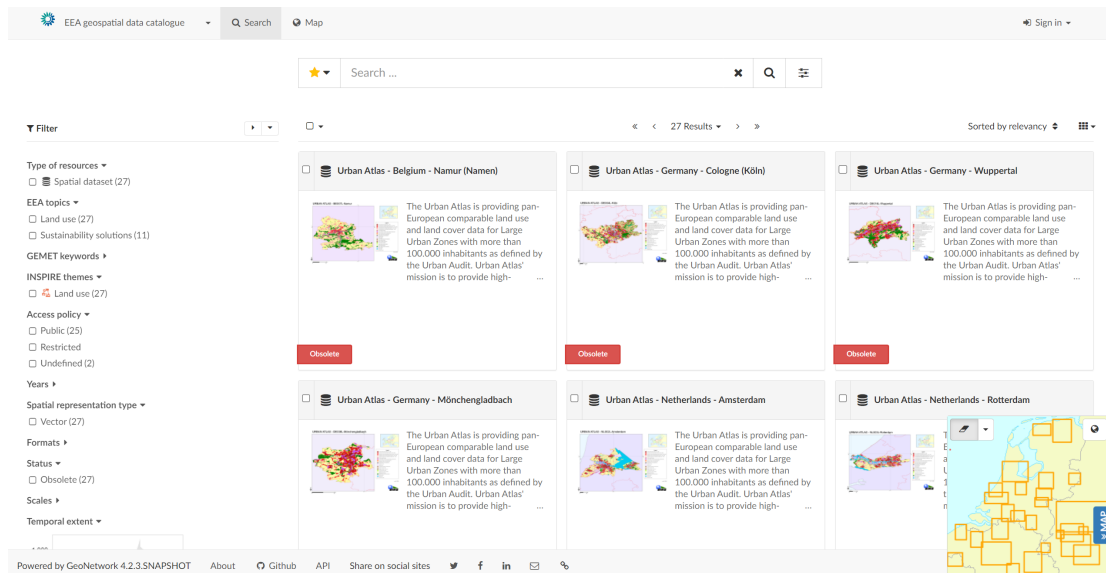


Figure 2.1: The GeoNetwork interface for the EEA metadata catalogue

In figure 2.1, the default UI for GeoNetwork is depicted. There are several features GeoNetwork provides to aid users in finding datasets. Firstly, users can search for relevant keywords in the search bar at the top. Secondly, GeoNetwork provides intricate filter capabilities, which can be accessed at the left of the UI. This panel allows users to filter search results based on dataset metadata, such as topics, data type and formats, temporal extent and access policy. Finally, users can filter by geospatial extent using the small interactive map in the bottom right. In this map, users can draw a rectangle and then filter by metadata records which are either contained in the rectangle or intersect with the rectangle. The map also displays the bounding boxes of the metadata records in the current search results. Besides this functionality, the small map does not provide much else, it is not possible to click on the bounding boxes to find out which metadata record is associated with it. Of course, the previously listed features are only available for GeoNetwork catalogues, and thus this UI is not available for other catalogues which are not GeoNetwork.

GeoNetwork provides another interactive map viewer, which is full-screen and dedicated to visualising the data in the datasets themselves. This map supports protocols such as WMS, WMTS or WFS. There is no built-in support for selecting data in this map or extracting only the selected data. Furthermore, while GeoNetwork allows users to download datasets as a file easily, there is no dedicated Jupyter integration in GeoNetwork.

2.3.4 Gap identification

In the previous subsections, we have looked at three categories of related work: geospatial data and Jupyter, subsetting systems and geospatial metadata catalogues. As shown, previous work in this field cannot entirely answer the research question as there are several main gaps in the previous work. Firstly, some solutions are only designed for large-scale proprietary datasets or specific catalogues. However, in this research, we want to be able to support all public datasets and make a solution that can be easily extended to support a large variety of formats. Secondly, most previous work considers only visualisation and analysis or only selection and extraction. Both need to be integrated into the tool to answer the research question. Finally, the majority of the existing work does not integrate well with the Jupyter ecosystem. The novelty of the new research lies in addressing the gaps identified in previous work. Specifically, the research aims to develop a solution that can be extended to support all public datasets and a variety of formats, integrate visualisation with selection and extraction, and, lastly, integrate well with the Jupyter

ecosystem.

Application design

In this chapter, we will specify the design of the application and elaborate on the various design choices. We will not yet examine the concrete technologies used to implement the tool. First, user stories and the tool's requirements will be defined. Then, the system architecture and all its components will be described and presented as a schema.

3.1 User Stories & Requirements

The objective of the application is to allow researchers to effectively browse metadata catalogues, browse datasets, select data and export this data to Jupyter. The objective will be divided into several user stories, each of which will contribute to the development of the application.

- As a researcher, I want to be able to browse metadata catalogues in the application so that I can quickly find relevant datasets for my research.
- As a researcher, I want to view detailed information about a dataset, so that I can assess its suitability for my research.
- As a researcher, I want to be able to preview the data within a dataset before using it in my research, so that I can determine if it meets my requirements.
- As a researcher, I want the ability to easily export selected a dataset to Jupyter Notebook, so that I can seamlessly integrate the data into my data analysis workflows.
- As a researcher, I want the ability to easily export a subset of a dataset to Jupyter Notebook, so I can easily visually choose the relevant data based on spatial position.

Based on the objective and the user stories, there are several requirements that the application should satisfy. The application should:

- effectively visualize the contents of geospatial metadata catalogues.
- support a wide variety of geospatial metadata catalogues.
- be able to visualize the information in the metadata records clearly.
- support the visualization of raster and vector datasets.
- be easily accessible for Jupyter users.
- allow the dataset to be easily imported into Jupyter.

3.2 System architecture

In this section, the design choices related to the overall architecture of the application will be discussed. Then, the overall system architecture will be illustrated in a diagram.

The first design choice we aim to address is finding a solution to browse and visualize meta-data catalogues effectively. The solution GeoNetwork provides allows users to search and filter datasets which are then displayed in a grid. In this grid, the titles and descriptions of metadata records are shown; however, this does not give users a clear indication of the spatial extent of the metadata record. This property of datasets can be of significant importance when a researcher selects a dataset.

Therefore, the solution that the application will use involves visualising the bounding boxes of the metadata records on an interactive map, resembling the small map integrated into GeoNetwork. Contrary to GeoNetwork, this map will be the primary display method of the metadata records and provide a higher level of interactivity. The shown metadata records will be based on the spatial extents of the current viewport of the interactive map and automatically updated when the user moves the map. Users can then click on bounding boxes and view the information contained in the associated metadata record.

Besides the map, a table containing the metadata record titles and other relevant information besides spatial extents will also be shown in the same interface. This assures users can also select datasets based on relevant information besides the spatial property of the dataset if they require it. With the combination of the map and table, we aim to provide a solution which allows users to effectively browse metadata catalogues.

Another design choice that must be made is the application's runtime environment. Based on the requirements, the application must be easily accessible for Jupyter users. This requirement leaves out a desktop application, as the friction this introduces makes this not easily accessible to Jupyter users. The other options for the environment are a web application or a Jupyter extension. Creating the application as a Jupyter extension would allow for some functionality otherwise impossible, such as the map viewer as a tab within Jupyter and access to the notebook the user is working in. On the other hand, making a standalone web application allows users to use the application with platforms other than Jupyter, such as native Python scripts, Jupyter Notebooks within Visual Studio Code or even other programming languages. The tool would still be easily accessible for Jupyter users, as both applications can run in the same browser.

Based on the requirements and the design choices made, the system architecture will primarily revolve around the web application.

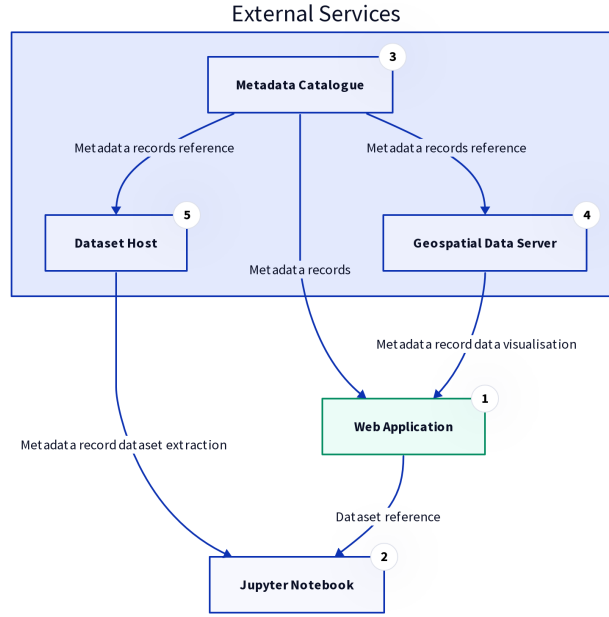


Figure 3.1: High-level architecture of the application and its external components

Figure 3.1 presents the architecture of the application and its relations with external components. Users connect with the metadata catalogue³ using the GUI in the application¹, information about the catalogue is loaded, and metadata records are queried to be shown on the interactive map and table. When the interactive map is moved, the bounding box is calculated, and the corresponding metadata records are queried from the catalogue³ again. Users can then click on a metadata record in the map or table to view all information stored in the metadata record. If the user thinks that this metadata record would be suitable for their use case, the user can choose to view the dataset on the interactive map using a geospatial data server⁴ that is defined in that metadata record. Then, the user can import the dataset into Jupyter² using the reference to the dataset host⁵ contained in the metadata record. Optionally, the user can also choose to only extract the features selected on the map.

Implementation

This chapter will focus on discussing the implementation of the application. We will begin by discussing the specific choices made during the implementation process, such as the selection of the map viewer library, the supported metadata standards and protocols, and the integration with Jupyter. We will revisit the architecture schema using these concrete technologies and examine a time sequence diagram that illustrates an example user interaction.

4.1 Technical choices

This section aims to explain the rationale behind the selection of specific technologies and how they contribute to achieving the desired goals. Furthermore, the design choices related to the implementation of various components and functionalities are discussed.

4.1.1 Map viewer

Four web mapping libraries have been reviewed in the preceding relevant work chapter. These are Leaflet, OpenLayers, Mapbox and Maplibre. For this project, several properties are especially important in a map viewer:

- It must support WMS, WMTS and WFS.
- It must be widely used and proven in the field.
- It must be actively maintained.
- It must be compatible with modern web technologies.

To compare the four libraries, we will look at these properties side-by-side. To indicate how widely used a library is, we will use the weekly NPM downloads metric, which should give a reasonable indication. We will use the date the last version was published to compare whether the libraries are actively maintained.

	Leaflet	OpenLayers	Mapbox	Maplibre
WMS support	Yes	Yes	Yes	Yes
WMTS support	Yes	Yes	Yes	Yes
WFS support	Plugin	Yes	No	No
Weekly NPM downloads	851,684	196,606	911,710	150,778
Last published (as of 16 May 2023)	6 months ago	4 days ago	1 month ago	1 day ago
Open source	Yes	Yes	No	Yes

Table 4.1: Comparison of most popular web mapping libraries

In table 4.1, a comparison of the libraries is shown. All libraries support WMS and WMTS protocols, but only OpenLayers natively supports WFS. Leaflet supports WFS through the plugin leaflet-wfst. However, this plugin was last published six years ago and is thus not considered actively maintained. While Mapbox and Leaflet have by far the most weekly downloads, OpenLayers and Maplibre are also frequently used. Except for Leaflet, all libraries are very recently updated. Even though Leaflet's last version was published six months ago, commits are still regularly being made on the git repository. Lastly, all libraries are open-source, with the exception of Mapbox.

Based on these properties, OpenLayers is deemed the most suitable for this project. OpenLayers' most significant advantage over other libraries is the wide range of advanced features OpenLayers supports, such as native WFS capabilities. In addition, the application we are creating is very comparable to GeoNetwork, which also makes use of OpenLayers as its built-in web mapping solution.

4.1.2 Metadata catalogue protocols

In the design chapter, some requirements for the program were established. One of those is: The application must support a wide variety of geospatial metadata catalogues. This is an essential requirement to base the supported metadata standards and protocols for the application on.

Several metadata protocols were reviewed in Chapter 2; these are CSW, GeoNetwork API, OAI-PMH and OpenSearch. In Chapter 2, the most popular metadata catalogues were also discussed: GeoNetwork, pycsw, GeoNode and CKAN. Therefore, to support a wide variety of geospatial metadata catalogues, we must thus choose a protocol that is at least supported by these three catalogues. As we have already seen, GeoNode and CKAN are both based on pycsw; thus, in order to find a protocol, the protocols supported by both pycsw and GeoNetwork be compared. In this case, all reviewed protocols except for GeoNetwork API are supported by both GeoNetwork and pycsw. Consequently, supporting one of CSW, OAI-PMH, or OpenSearch will allow the application to support both GeoNetwork and all pycsw-based catalogues and, thus, a wide variety of geospatial metadata catalogues.

One additional requirement for these protocols is that a comprehensive filtering system must be available, allowing for spatial and temporal filters.

According to the OAI-PMH specification, OAI-PMH supports selective harvesting. This allows users to filter results based on given timestamps to return only records that are created, deleted or updated within this date range. Selective harvesting additionally allows for filtering based on record sets, which are used to group records into categories. While these capabilities give users quite some control when searching for datasets, no spatial filters are natively supported by OAI-PMH, which disqualifies OAI-PMH as a candidate for the application.

OpenSearch by itself does not allow for advanced filtering or spatial/temporal awareness. Therefore, the OGC created the OpenSearch Geo and Time Extensions, which extend OpenSearch by adding more capabilities. While pycsw implements both the Geo and Time extensions, GeoNetwork does not and only allows for search based on a bounding box, location name and geometry, which is not granular enough for our use case. As we cannot support only pycsw and not GeoNetwork, OpenSearch is also disqualified.

This leaves CSW as the only remaining candidate. As already established, CSW supports a very rich query language, which supports text-based search, spatial and temporal search, and filtering based on other custom properties. CSW is also created specifically for the use-case of this application, a metadata catalogue protocol for geospatial data. Furthermore, CSW is created by the OGC, which is the predominant organization for geospatial protocols and standards; thus, supporting a protocol created by the OGC ensures wide compatibility with the ecosystem besides GeoNetwork and pycsw. For these reasons, CSW will be the metadata catalogue protocol supported by the application.

4.1.3 Metadata standards

CSW returns metadata records, and the application must understand the format of these records in order to show the data in the user interface and extract the bounding boxes. According to the CSW standards, the metadata records exposed by CSW can be in any XML-based format. In Chapter 2, the most well-known XML-based metadata record standards have been reviewed. These are ISO 19115, Dublin Core, INSPIRE Metadata and CSDGM. To allow the application to support the most possible catalogues, a standard must be chosen which is widely used.

CSW services must minimally support Dublin Core; furthermore, if no specific standard is specified in the request, by default Dublin Core is returned. The Dublin Core schema specified in the CSW specification is extended from standard Dublin Core to define the bounding box of the metadata record. Besides this, Dublin Core is a simple format which only exposes minimal information about the metadata record.

INSPIRE metadata is based on ISO 19115; consequently, supporting ISO 19115 makes adapting INSPIRE metadata trivial. CSDGM has been deprecated, and the organization behind the standard currently recommends using an ISO standard instead of CSDGM.

ISO 19115 is the most widely used standard in the geospatial community. Compared to Dublin Core, it contains much more information which can be of importance when selecting a dataset. As GeoNetwork and pycsw have great support for ISO 19115 and recommend ISO 19115 for spatial resources [17], ISO 19115 will be used as the main metadata standard for the application.

4.1.4 Client libraries

The application interacts with several external services: CSW, WFS, WMTS and WMS. To interact with these services, custom implementations to interact with the protocols can be created or preferably, libraries can be used to handle this for us.

For the mapping library, OpenLayers was chosen, which already has support for the WFS, WMTS and WMS services. As such, OpenLayers can be used for this; a custom implementation or separate library is unnecessary.

On the other hand, OpenLayers does not support CSW, so another library is needed to interface with a CSW server. For this purpose, the libraries `ows.js` [40], `csw-client` [11] and `@map-colonies/csw-client` [1] have been identified. The libraries `ows.js` and `csw-client` are based on the `Jsonix` [26] library to parse the schema, while `@map-colonies/csw-client` is based on `inspire-parser` [23]. The `inspire-parser` library was last published five years ago and is not maintained anymore. Furthermore, while it can parse CSW responses and Dublin Core metadata records, it is not able to deserialize complete ISO 19115 metadata records. `Jsonix` has seen much more adoption than `inspire-parser` as it is made for geospatial data and any XML serialization/deserialization in JavaScript. `Jsonix` is schema aware, as a consequence, it can create a much more useful deserialized object than a deserializer which does not know the schema. However, this also means it depends highly on the schemas, which must be registered in the library before the deserialization. If the parser encounters an element which is not preregistered, the parser will error; this functionality is not desired. `Jsonix` does not have any TypeScript integration for the schema, which makes working with the metadata records very error-prone. Furthermore, `Jsonix` is not maintained anymore, as the last version was published three years ago.

Because of these issues, no viable CSW library has been identified. Consequently, for this application, a custom implementation will be made for the ISO 19115 parser, the CSW client API and the CSW request/response parser.

4.1.5 Cross-Origin Resource Sharing

Cross-Origin Resource Sharing (CORS) [9] is a security feature of browsers which allows websites to share data with other websites through HTTP requests. By default, the browser protects users from websites that try to read potentially sensitive data from other websites on behalf of the user. Web servers can decide whether to allow HTTP requests originating from other websites on a per-request basis.

For this project, a problem arises when web servers accessed by the application do not allow for Cross-Origin Resource Sharing, as the browser will then block the HTTP requests. By default, many CSW, WFS, WMS and WMTS servers have CORS enabled, but as the OGC specifications do not require this property, a service can disable CORS and still qualify as an OGC-compliant service. Therefore, to support any OGC-compliant service, a solution for non-CORS services needs to be implemented in the application. For this purpose, several workarounds have been identified.

The first workaround is changing the environment of the application. As CORS is a mechanism that is only obeyed by a web browser, changing the application's runtime environment would solve the CORS issue. Two possibilities for a new runtime for this application would be as a standalone desktop application or as a Jupyter extension. Creating a standalone desktop application would take away from the accessibility of the application, as users would then need to download and install the application, which introduces friction and requires more effort from the user.

A second workaround would be to create a browser extension. This browser extension could then override the CORS protection of the browser so the browser can make requests to services which are not CORS enabled. While installing browser extensions generally requires less effort than downloading and installing a new application, this would still inconvenience users. This would also require creating multiple extensions to support Chromium, Firefox, Safari and other browsers the user would want to use, which can take a lot of effort and maintenance.

The final solution would be to create a proxy server. This is a common workaround to CORS issues in the browser as it requires the least effort from the user. The proxy solution introduces a layer of indirection to each request the application wants to make to an external server. Instead of being directly sent to the external server, the request is sent to a proxy server. This proxy server can then make the request to the external server, as the proxy server does not have to comply with the CORS rules imposed by the external server. This setup requires the least effort from the user's perspective but does introduce additional infrastructure, cost and maintenance. Moreover, the usage of a proxy server will decrease the application's performance depending on the network latency between the client, proxy server, and external server. To minimize this latency, the infrastructure the proxy server is deployed on must be considered.

Depending on the users' location, the proxy server could be deployed to a single server closest to the users if the users are all in the same proximity. If this is not the case, the latency between the users and the proxy server would significantly decrease the performance of the application. Replicating the proxy server across multiple regions would solve this issue, but this would greatly increase the complexity of the infrastructure.

In recent years, the serverless computing paradigm has emerged to deploy servers close to the end-user [4]. Using serverless, application code is deployed across many regions, ensuring a proxy server is always close to the end-user. Even more recently, multiple edge runtimes such as Cloudflare Workers, Vercel Edge Functions and Deno Deploy have emerged. Most serverless runtimes are based on the Node.js runtime; edge runtimes use a more minimal engine. This makes deploying on edge runtimes more performant and cost-effective, and solves the problem of cold starts, which serverless functions suffer from.

Three solutions have been reviewed to address the issue of interacting with OGC services that have CORS disabled. Firstly, the runtime environment of the application could be changed to a desktop application or Jupyter extension, which solves the problem but introduces friction for the user. Secondly, an extension could be created that overrides the CORS settings from requests from our website; however, this also requires the user to do more work. Finally, a proxy server could be used, which complicates infrastructure, introduces cost and increases latency but, most importantly, does not require any work from the user.

For this reason, we will introduce a proxy server in the infrastructure for the application. By leveraging serverless computing and deploying to an edge runtime, the complexity of the in-

frastructure, the cost and the end-user latency will not be increased much while ensuring the application remains highly accessible.

4.1.6 Jupyter integration

An important function of the program is the integration with Jupyter. The application must support a way to seamlessly export the data to Jupyter in an efficient way that takes as little effort as possible for the user.

The most trivial way to offer this functionality would be to allow the user to download the dataset and then let the user upload it to Jupyter. This would allow for very flexible and dynamic use cases; for example, users could use the program in environments other than Jupyter. However, it would also not be efficient and seamless, as this workflow is not tailored directly to Jupyter.

Another method to provide this functionality is dynamically generating a Python script which the user can easily copy and paste into Jupyter. This functionality would be specific to Jupyter, which allows for the optimal workflow for Jupyter users. The Python script generated by the application should download the file to disk if it does not exist on the disk already, or if it does exist, read it from the disk and load it into memory.

While the application is primarily tailored for Jupyter users, implementing both exporting methods would allow for seamless integration with Jupyter and the most dynamic use cases the user can have. For this reason, both methods will be implemented in the application.

4.2 Technical Architecture

Having discussed all the considerations regarding the program's implementation, we can now visualize the architecture, including the implementation details.

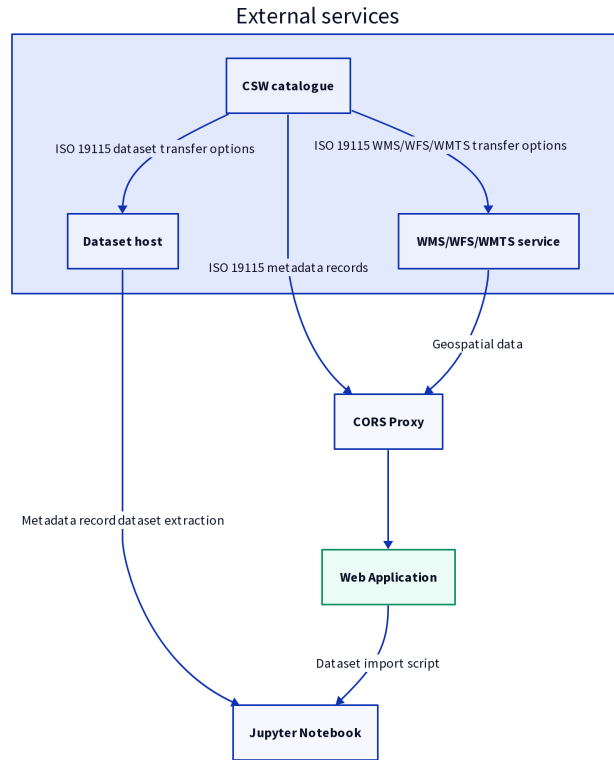


Figure 4.1: The architecture of the implementation of the application. The metadata catalogue is a CSW service and all records are structured according to ISO 19115. All traffic from the application to external services is routed through the proxy.

In figure 4.1, the architecture of the application implementation is seen. The metadata catalogue is a CSW service, and the metadata records are ISO 19115 records. All network requests from the application to external services are proxied through the CORS proxy, which is a serverless edge function. The application generates a Python script, which is copied to the clipboard so the user can then paste the script into the Jupyter environment.

To illustrate the interactions between the components shown in figure 4.1, a time sequence diagram will be shown of an example user interaction. From the perspective of the user, the example interaction will look like this:

1. The user inputs a CSW endpoint in the application.
2. The application shows the metadata records inside the CSW service visualized on a map.
3. The user selects a metadata record.
4. The application shows the detailed information stored in the record.
5. The user selects a WFS/WMS/WMTS service associated with the record.
6. The application lists metadata inside the service.
7. The user selects a feature/layer.
8. The application shows this feature/layer on the map.

9. The user clicks export to Jupyter.
10. The export script is copied to the clipboard, and the user can now paste it into a Jupyter cell.
11. The user runs the Jupyter cell.
12. The data is downloaded to the disk and memory.
13. The user can now use and analyze the data in Jupyter.

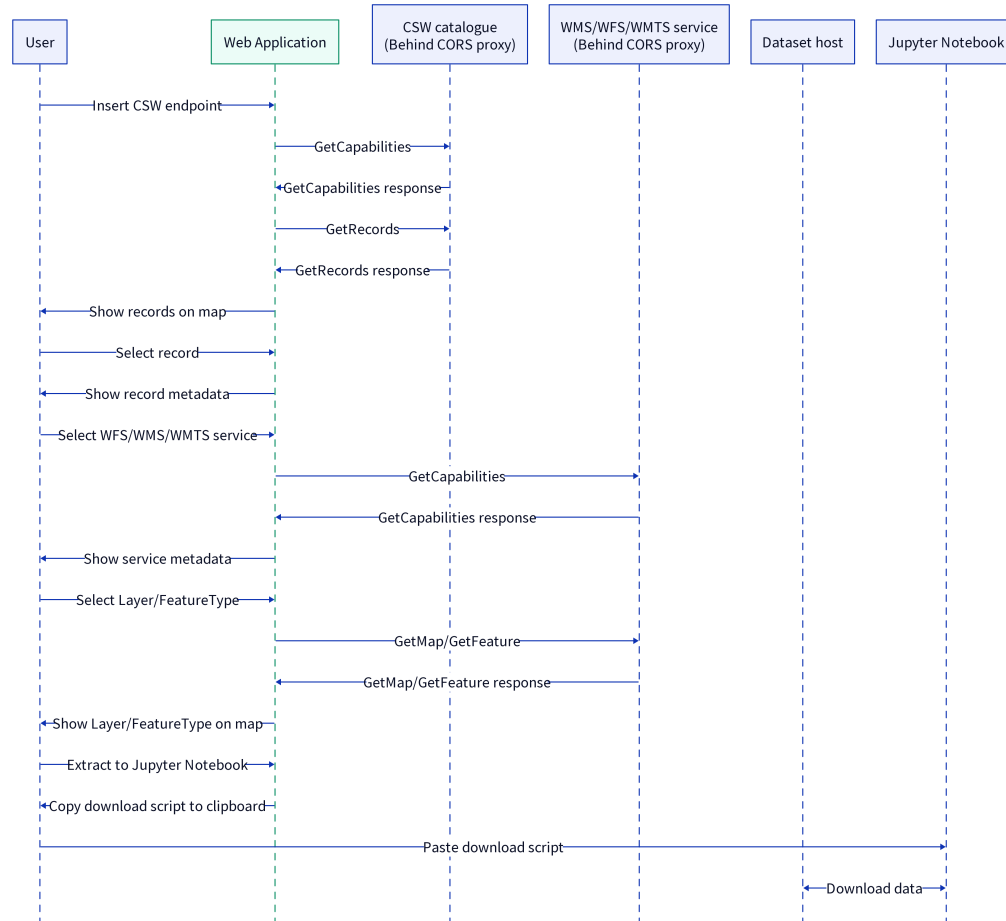


Figure 4.2: Time sequence diagram of interactions between system architecture components for the example user interaction. The CSW proxy is not shown in the diagram for conciseness; however, all interactions with the CSW catalogue and WMS/WFS/WMTS service go through the proxy.

In figure 4.2, the time sequence diagram displaying the interactions between components of the system architecture is shown. When the user submits a CSW endpoint, the application makes a GetCapabilities request to the CSW catalogue, proxied through the CORS proxy. The response of the GetCapabilities function is parsed, which the application then uses to display the name of the CSW service. When the map viewer loads in, the web application sends a GetRecords request to the CSW service with a filter so only the records within the map's bounding

box are returned. The boundary boxes of these records are then drawn on the map. Every time the user moves the map, new records are queried again for the new map bounding box. The queried records are also paginated in order to not put too much information on the map and to reduce the load on the CSW service.

The user then selects a record on the map; when a record is selected, the metadata is shown in the application. In this metadata, ISO 19115 stores associated WFS/WMS/WMTS services, which are also shown in the application. The user can then decide to view such a service by selecting one. Then, the capabilities of the service are queried again using `GetCapabilities`. This allows the application to display information about the service in the UI, under which layers/feature types.

When the user clicks on a layer/feature type, the application sends a `GetMap/GetFeatures` to the service, and the response is rendered on the map. Finally, the user exports data from the selected service or another associated data source stored in the metadata record and chooses to export it to Jupyter. The application generates a Python script, which is copied to the clipboard. The user pastes this script into a new Jupyter cell and runs the cell. The data is downloaded, and the user can now use this data in their analysis.

4.3 Current status

The application is available online at <https://thesis-app.vercel.app/>. Furthermore, the application's source code is open-source and available at <https://github.com/vigovlugt/thesis-app>. All features discussed in this chapter are implemented in the application, except for WMTS support and the CORS proxy, due to time constraints.

Demonstration

In the previous chapter, the technical interactions of the application and external systems were shown, according to an example user interaction. In this chapter, we will demonstrate this example user interaction from the user's perspective, showcasing the application and the user interface. The chapter is split up in the general stages the user will experience while using the application.

5.1 Selecting an CSW endpoint

In this section, the following step of the user interaction will be shown: The user inputs a CSW endpoint in the application.

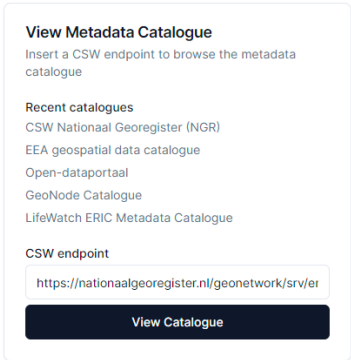


Figure 5.1: CSW endpoint insertion form, showing CSW input and recently browsed catalogues

In figure 5.1, the CSW endpoint selection screen is shown. When a user opens the application, this is the first screen they will see. The user can insert any CSW endpoint URL in the input

or select a CSW endpoint they have used previously. Upon submission of the form, the user will navigate to the next screen of the application.

5.2 Browsing the metadata catalogue

In this screen, the user will perform the following steps of the interaction:

- The application shows the metadata records inside the CSW service visualized on a map.
- The user selects a metadata record.
- The application shows the detailed information stored in the record.
- The user selects a WFS/WMS/WMTS service associated with the record.

The screenshot shows the 'Parkeerzones' metadata catalogue view. The interface is divided into several sections:

- Search and Filter:** A search bar labeled 'zones' and a 'Topic code' section with various filters like Farming, Biota, Boundaries, etc.
- Map:** A map of the Netherlands with several bounding boxes overlaid, indicating the spatial extent of the metadata records.
- Table:** A table listing metadata records with columns: Title, Status, Date, and Categories. The first three records are highlighted in blue, corresponding to the bounding boxes on the map.

Title	Status	Date	Categories
Gezondheidskaart GES Geluid spoorwegen Lden 2013	Completed	10/6/2021	Environment Health
Gezondheidskaart GES Geluid industrie	Completed	10/6/2021	Environment Health
Zones rond wierden en wierdedorpen (Omgevingsvisie)	Completed	7/4/2019	Planning Cadastre Society
- Details Panel:** A panel on the right showing detailed information for the selected record, including 'Technical specifications' (Representation type, Format, Scale, CRS, Publication, Creation, Language, Resource identifier) and 'Resource constraints' (Niet geschikt voor commercieel gebruik).

Figure 5.2: Metadata catalogue view when viewing Dutch metadata catalogue 'Nationaal georegister'

Figure 5.2 shows the screen the user sees after submitting a CSW endpoint. In this screen, the user can browse the metadata records in the catalogue. On the left side, the user can search for a keyword or filter the records based on topic or resource type. The filtered metadata records are shown in the table at the bottom, and the corresponding bounding boxes are shown on the map. The user can pan and zoom the map to search for records in another spatial region. The currently selected metadata record is highlighted in the table and shown in blue on the map.

The details of the selected metadata record are displayed in the right pane of the view. Here the user can view information like the title, categories, keywords, technical specifications, resource constraints and contact information. Using this information, the user can decide whether or not this dataset can be of use for their use case.

Beneath this information, the linked resources of the record are shown, categorized as services, files and links. The services list contains the WFS/WMS/WMTS services associated with the record. When the user selects one, they will transition to the next screen.

5.3 Service view

In this view, the following interactions take place:

- The application lists metadata inside the service.
- The user selects a feature/layer.
- The application shows this feature/layer on the map.

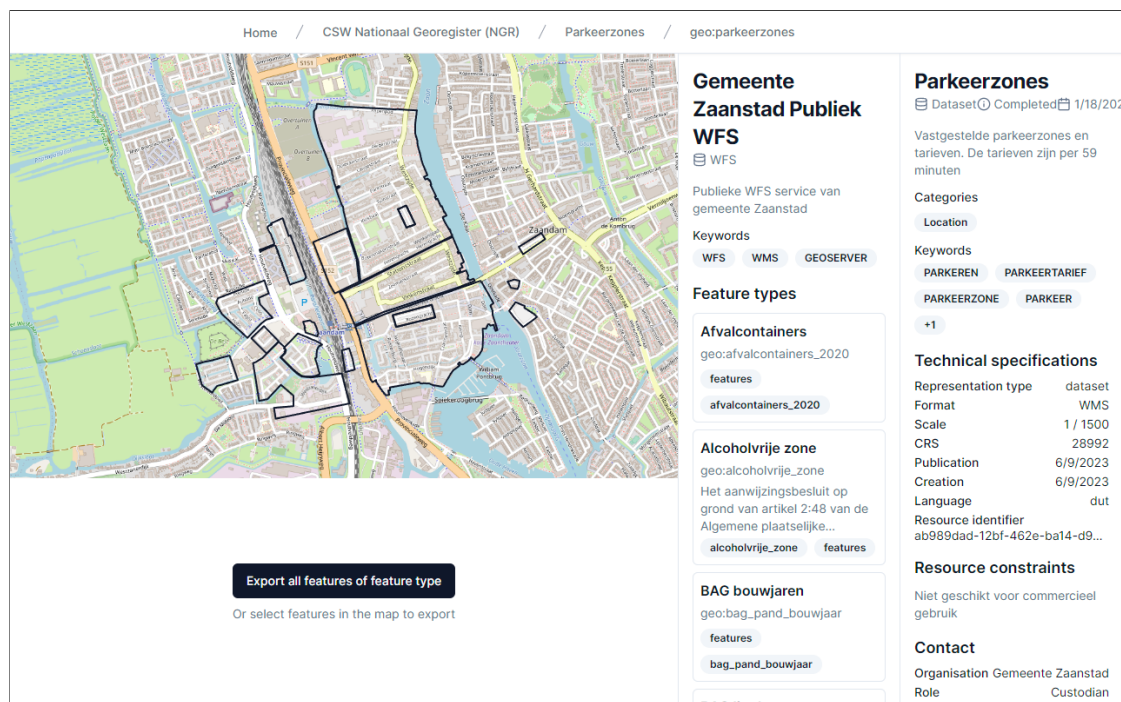


Figure 5.3: Service view when viewing WFS service of 'Parkeerzones' metadata record of metadata catalogue 'Nationaal georegister'

In figure 5.3, the service view is shown. On the right side of the screen, you can see that the selected metadata record information panel is still shown; this ensures the user can easily switch between services associated with that record. Left to this panel, the service information panel can be seen, which shows details about the service and shows the feature types, or layers, depending on whether the current service is a WFS or WMS/WMTS service. The user can switch between these feature types or layers, and the data within the currently selected feature type or layer is shown on the map.

5.4 Exporting data

This final stage takes place in the same view as the previous stage. The following interactions are performed:

- The user clicks export to Jupyter.
- The export script is copied to the clipboard, and the user can now paste it into a Jupyter cell.

To export data from the dataset to Jupyter, the user can export any downloadable resource which is associated with the metadata record. If a WFS service is selected, the user can also export the data in the current feature type. Furthermore, the user can select a subset of features within the map, and then export only those features.

5.4.1 Exporting a downloadable resource

If the metadata has associated downloadable resources, users can click these in the metadata panel to open an export dialog.



Figure 5.4: Resource export dialog for resource 'CSV' of metadata record 'Parkeerzones' of metadata catalogue 'Nationaal georegister'

Figure 5.4 shows this dialog. The user can choose to export the resource to Jupyter or download it as a file. When exporting to Jupyter, the application generates a Python script which will download the file using the requests library or read it from disk using the built-in os library if the file already exists.

As these resources can be of any format, this script must also work for any format.

5.4.2 Exporting a feature type

If the user is inspecting a WFS service, they can also export all data within a feature type.

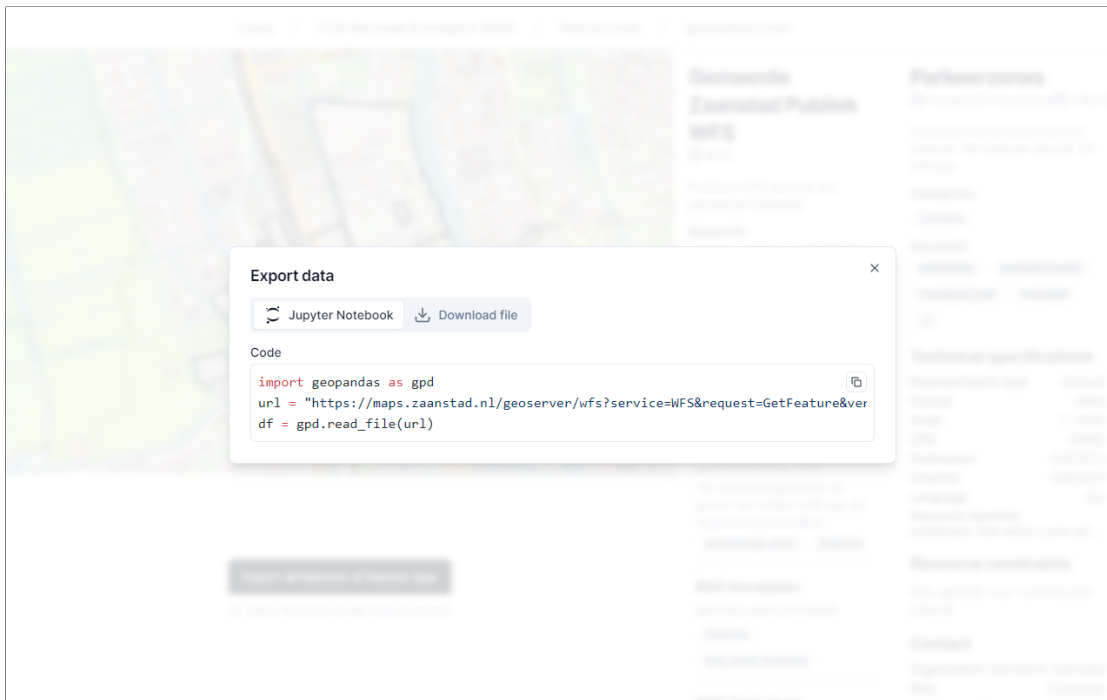


Figure 5.5: Feature type export dialog when viewing WFS service of 'Parkeerzones' metadata record of metadata catalogue 'Nationaal georegister'

In figure 5.5, this modal is shown. Unlike exporting a resource, this data will always be in the GeoJSON format. Therefore, the export script uses GeoPandas, a popular Python package for working with geospatial vector data.

5.4.3 Exporting a subset of features

Finally, the last export method allows the user to export a subset of features.

Home / CSW Nationaal Georegister (NGR) / Parkeerzones / geo:parkeerzones

Gemeente Zaanstad Publiek WFS

Publieke WFS service van gemeente Zaanstad

Keywords: WFS, WMS, GEOSERVER

Feature types

Afvalcontainers
geo:afvalcontainers_2020
features, afvalcontainers_2020

Alcoholvrije zone
geo:alcoholvrije_zone
Het aanwijzingsbesluit op grond van artikel 2:48 van de Algemene plaatselijke...

BAG bouwjaren
geo:bag_pand_bouwjaar
features, bag_pand_bouwjaar

Parkeerzones

Dataset Completed 1/18/2023

Vastgestelde parkeerzones en tarieven. De tarieven zijn per 59 minuten

Categories: Location

Keywords: PARKEREN, PARKEERTARIEF, PARKEERZONE, PARKEER

Technical specifications

Representation type	dataset
Format	WMS
Scale	1 / 1500
CRS	28992
Publication	6/9/2023
Creation	6/9/2023
Language	dut
Resource identifier	ab989dad-12bf-462e-ba14-d9...

Resource constraints

Niet geschikt voor commercieel gebruik

Contact

Organisation Gemeente Zaanstad

Role Custodian

gebied	tarief_per_uur	tarief_per_dag	maximale_parkeerduur	tijden_betaald_parkeren
RB	€ 3,45	nvt	onbeperkt	maandag t/m zaterdag van 9
OW	€ 3,45	nvt	onbeperkt	maandag t/m zaterdag van 9
OW	€ 2,90	nvt	onbeperkt	maandag t/m zaterdag van 9
ZO	€ 2,90	nvt	onbeperkt	maandag t/m zaterdag van 9

Export selected features

Figure 5.6: Selecting features on the map when viewing WFS service of 'Parkeerzones' metadata record of metadata catalogue 'Nationaal georegister'

As shown in figure 5.6, users can select features in the map by clicking on individual features, or pressing shift and dragging a box over the features that the user is interested in. The properties of selected features are also shown in the table beneath the map. Clicking the export selected features button will then open a dialog.

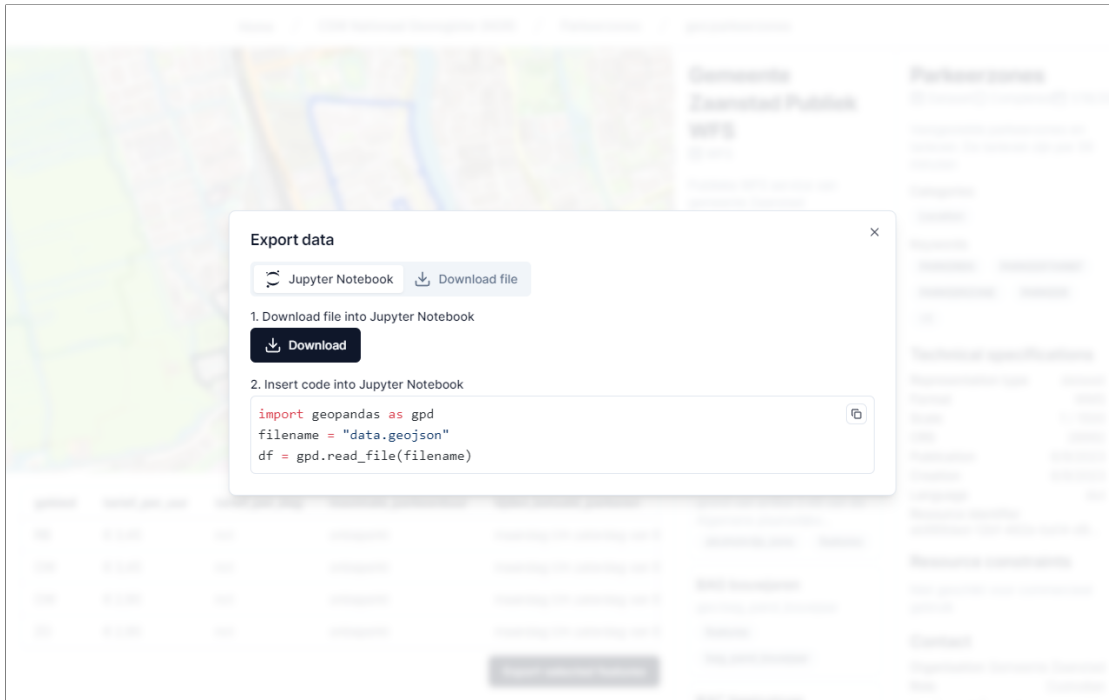


Figure 5.7: Selected features export dialog

In figure 5.7, this dialog is shown. To export this to Jupyter, the dialog instructs you to first download a file and then insert a script which reads the file with GeoPandas again. For this export method, the user has to download a file, which, unfortunately, takes more effort than the other steps. This is the case, as the other methods provide an export for data which is hosted elsewhere, but this method is specifically tailored to this user. This means that the data must also be specific to the selection of the user, which is why the application must create a file with this data. All in all, while this method takes more effort from the user, relative to the other methods However, it also provides a very easy and quick way to create a subset based on spatial properties visually.

Experimental results

In this chapter, a usability test will be conducted. In this test, participants will complete several tasks that represent how data researchers would use the tool. The tasks' success rate is measured, along with the time it takes to complete them.

In the first task, participants are asked to view the catalogue of a given CSW endpoint. The given CSW endpoint is that of the *Nationaal Georegister* [32]. This catalogue has been chosen as it contains numerous metadata records with WFS services attached, thus being able to show and test the application's full capabilities.

Before the second task is conducted, a small introduction to the catalogue page is given. This is done to give participants the main idea of the features available on this page, and how it should be operated. The main objective of this usability test is not to discover whether or not the application is immediately understandable for first-time users but to see whether or not users can efficiently use the application to complete the tasks the application is designed for.

After this introduction, the second task is given to the participants, which asks participants to find a dataset with information about historical mills in North Holland, containing a WFS service. This specific dataset has been chosen as there are several datasets about mills in the catalogue; thus, this tests whether users can effectively compare similar datasets. The test also specifically states a region, as finding datasets based on spatial properties is an important function of the application.

In task three, participants must preview the WFS service to see the data in the dataset. This test examines another important feature of the application, the ability to preview datasets. After this test, another small introduction is given to the WFS preview page, explaining the purpose of each panel in the UI, and explaining that users are able to select features that they are interested in exporting.

In tasks four and five, which are the final tests, the exporting functionality of the application is tested. Task four asks the user to select the currently selected feature type of the WFS service, which must then be imported into Jupyter to finish the task. In task five, users are asked to select five features and to import those into Jupyter too. These tasks test the final but crucial features of the application, namely the exporting capabilities, the integration with Jupyter, and the subset creation. After these tasks, the usability test is completed.

Following the usability test, an impression test is performed with the goal of measuring the impression the application left on the participants. This test is derived from a study which was designed by Microsoft for evaluating desirability in a usability lab setting [5]. In the impression test, participants are given a list of words, and they are tasked to circle five words which they feel best describe their experience using the site. The original study included 118 words that

describe emotions or impressions; however, 118 words is too much to ask a participant to look through, which is why for this test, the list was reduced to 25 relevant words. A major part of the original words regarded appearance, which is not a property that is especially important for this application which is primarily focused on usability and efficiency. Therefore, the 25 chosen words mainly considered accessibility, complexity/ease of use, efficiency and valuableness of the application. The full list of the 25 words can be found in Appendix A.2.

Finally, an open-ended question is asked to gather any additional feedback from the participants. As these participants are professionals in their field, this feedback is important to collect, so the application can be proven useful or ineffective. The full form of the test can be found in Appendix A.2

6.1 Results

The test was completed by 4 participants, these participants are all professionals in the field of geospatial data.

Task	Completion rate	Average duration (s)	Durations (s)
1	4/4	38	137, 5, 5, 4
2	3/4	57	120, 42, 10
3	4/4	4	10, 4, 2, 1
4	4/4	105	180, 120, 60, 60
5	4/4	58	120, 60, 30, 20

Table 6.1: Completion rate and average duration of the usability test tasks (n=4)

Table 6.1 presents the results of the tasks. Overall, all tasks were successfully completed by all participants except for task 2, which one participant could not complete. The participant submitted the reason why they could not complete task 2: "Couldn't find the molens dataset. A lot of modern windmills datasets get in the way. I should've used the filters". The average duration of all individual tasks is under one minute, except for task 4. The average duration of all tasks combined is 4 minutes and 22 seconds.

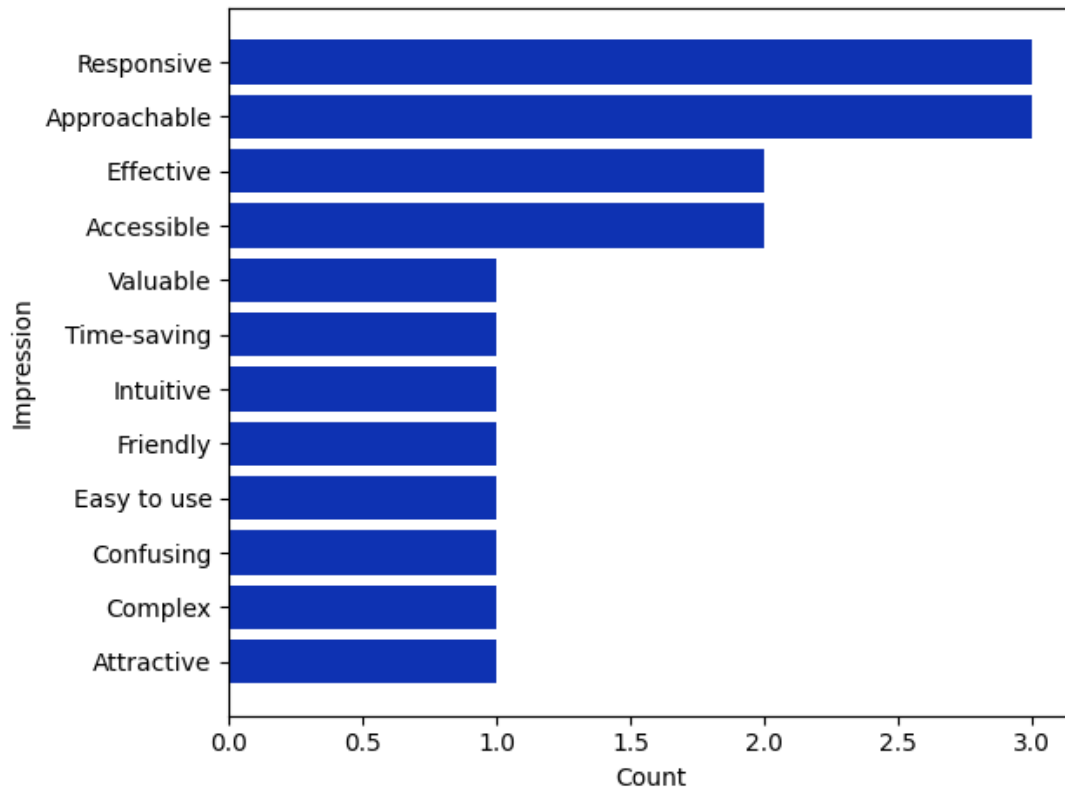


Figure 6.1: Results of the impression test (n=4)

Figure 6.1 shows the result of the impression test. Participants were asked to choose 5 impressions, however, 2 users only chose 4 impressions, which means that there are 18 impressions given in total. Out of these 18 impressions, 16 impressions are positive, and 2 are negative (Confusing, Complex). 75% of the participants found the application to be responsive and approachable, and 50% of the participants found the application effective and accessible.

Feedback 1	It's quick, looks easily expandable with other features, it has it's use and would be great as a Jupyter plugin! Some remarks: When selecting features on the map, It's hard for the user to see how to do this. We usually have some UI buttons enabling a 'select' mode which allows you to draw areas of interest.
Feedback 2	Looks really impressive! Well done. Maybe the extraction/subsetting would be even more user-friendly if it wouldn't require downloading the geojson manually but if it could generate a link that could be performed via a GET request inside the notebook. This way you can instantly use it inside a notebook.

Table 6.2: Miscellenous feedback and suggestions of usability test

Out of the 4 participants, 2 have submitted additional feedback for the application, which is shown in table 6.2.

Discussion

In this chapter, the results of the usability and impression test will be discussed. Furthermore, we will reflect on the choice of picking CSW as the metadata catalogue protocol. Finally, the ethical considerations of the application will be discussed.

7.1 Usability & impression test

All participants were able to complete the tasks in the usability test, except for one participant in task 2, where participants were instructed to find a specific dataset using the application. This participant mentioned that they did not use the filters while searching for the dataset, which resulted in too many datasets to look through in order to find the right one. Using the filters to search for the dataset was indeed the right way to complete this task, and the intended way how the application should be used. Other participants were able to use the filters to complete this task.

Furthermore, the average duration reported for each task falls in line with what would be expected for these tasks. Only task 4 took longer than anticipated, with an average completion time of 105 seconds. In this task, participants were asked to export the dataset to Jupyter, which has to be a quick and efficient action, as this is instrumental for the application. What could explain this long duration is that participants were instructed to open a Jupyter or a Google Colab notebook in the description of task 4. However, the usability test was unclear whether to do this before or during the task. This would mean that the users might have counted this in the time it took to complete the export, which would explain the reported extended durations. This is a failure of the usability form; in a future test, it should be made clearer that opening the notebook should not be counted towards the duration of the task.

The impression test resulted in a high number of positive expressions, with the most common being responsive, approachable, effective and accessible. Accessibility was an important factor when designing the application, which influenced many significant decisions, such as the application being available in the browser. Moreover, the reason the application was created was to make the research process more effective; therefore, it is affirming to see 'effective' back in the impression test. However, in order to make a definitive statement about the effectiveness of the application, the test should be repeated, but this time without using the application. This would allow for a comprehensive comparison between the effectiveness of the workflow with and without the application. Due to time constraints, this will be left for future work.

The impression test also received two negative expressions: confusing and complex. The workflow the application aims to streamline is inherently complex, therefore, there will always be some amount of complexity reflected in the application. In the form, brief explanations of the various UI screens were presented to the participants in order to make the first impression of the application less confusing. A way this can be made more clear to the user is to create a tutorial video in which the example user interaction is shown, which gives an explanation of how

the application works. Even better would be to create an interactive tutorial in which the user is instrumented to walk through the example user interaction. Both these solutions would help with the confusion of first-time users, and possibly help users perceive the application as less complex.

In the first feedback, a participant suggested two features, clarifying the feature selection functionality by adding dedicated buttons in the UI and creating a Jupyter plugin for the application. Adding dedicated buttons for feature selection would definitely help users identify the box selection feature, and other selection buttons could also be added, for example, selecting features within a radius or selecting all features within a country/state/city border. In section 3.2, the decision was made to create a browser-based application instead of a Jupyter plugin, in favour of accessibility for non-Jupyter users. However, this decision also made it trivial to adapt the application to a Jupyter plugin, as Jupyter plugins can be created from web applications. The second feedback received brings up the process of exporting selected features, which currently involves first downloading a client-side generated file and a Python script. In the feedback, a suggestion is made to remove the downloading of the file in order to speed up the process. The reasoning behind why a file needs to be downloaded is discussed in section 5.4.3: as only a subset of features is needed, we create a new file which contains only this subset. Another possibility would be to download the entire file in Jupyter and create a subset dynamically using the ids of the selected features, which we can store in the script. This way, the user would not need to download a file, but the script would be much more involved and unnecessary data would need to be downloaded the first time the script is run, which may or may not be what the user wants. Both methods have tradeoffs, it could be interesting to let the user choose which method they prefer.

Overall, the results of the usability study are positive and suggest that there is value in the application. However, a participation count of 4 is not enough to make definitive statements about the application. On the other hand, the participants are very experienced in this field, which makes the results of the study more credible and high-value. In future work, the study should be expanded to more participants and include a comprehensive comparison with the workflow without the application.

7.2 CSW & metadata catalogue usage

In this section, we will reflect on the choice of CSW as the metadata catalogue protocol. Furthermore, we will discuss how the method some organizations use and store data in metadata catalogue protocols worsens the user experience of utilizing the applications.

The application is tightly integrated with CSW, which means that the good and bad parts of CSW are reflected in the application. Some catalogues using CSW are very slow, some CSW catalogues take more than 5 seconds to retrieve 10 records in a geographic region. This means that the user has to wait 5 seconds each time they move the map, which makes the application feel very unresponsive. While this is not a problem with the CSW protocol itself, it does have a big impact on the application. Furthermore, searching for records using CSW does not work perfectly. For example, in a CSW catalogue which contains the record "Molens in 1812", the search query "Molens 1812" retrieves this result, but the exact title as query: "Molens in 1812" does not.

How organizations use metadata catalogues, greatly impacts the experience of the researcher using it. The usage of catalogues can vary greatly between organizations. For example, some organizations carefully assign categories and keywords to records, but some do not. Some organizations do not assign boundary boxes to records, or these boundary boxes do not accurately represent the data. Many records do not have a related WFS/WMS service in their metadata, which means that the data cannot be previewed. This reduces a lot of the value that the application can provide. Some records do not contain a link to directly download the dataset, instead redirecting you to a dataset landing page where users can then download the data. Consequently,

the integration with Jupyter is not possible as this needs a direct link to the dataset.

7.3 Ethical considerations

Besides the technical and usability considerations, it is also important to review the ethical considerations of the application. No user data is stored in the application, which means that the privacy and protection of user data is not a consideration of the application.

The application makes significant use of external data, which brings some ethical concerns regarding intellectual property rights and transparency towards the owner of the data. As discussed, many catalogues do not directly link to the final download link of datasets but instead, link to a download page where the user can then download the data. As said, this means that the integration with Jupyter does not work, but on the other hand, data owners gain exposure and greater control over their datasets. There are important technical and ethical tradeoffs between providing an effortless experience for the user through direct download links, and the control and exposure of the data source through custom download pages. A solution where both parties receive the best of both worlds could be looked for in future work.

Public datasets also come with several ethical concerns, such as the accuracy, confidentiality and reliability of the data. Besides these concerns, there are also concerns around the usage of the data, such as responsible use or resource constraints which the users of the dataset are responsible for. These are significant concerns; however, the application is not directly responsible for these datasets or users' usage of them. The application cannot guarantee the accuracy, confidentiality and reliability of external public datasets or ensure that users use the datasets responsibly. What is the responsibility of the application, however, is transparency. It is crucial that the application shows the information in the metadata records clearly, comprehensively and unaltered, so the user is actively informed about the accuracy, reliability, and usage constraints of the dataset. The application should also disclose a link to the metadata record in the external source, to allow the user to access and verify the information from the source of the data.

Ultimately, the application's role regarding the discussed ethical considerations lies in promoting transparency and providing users with the necessary information to assess and understand the datasets they are working with, empowering users to make responsible, informed and ethical decisions.

Conclusion

In this thesis, we have addressed the research question: How to enable data science researchers to effectively browse, select and extract geospatial data from data catalogues via a Jupyter environment? First, we have reviewed the theoretical background, related technologies and state-of-the-art of geospatial metadata catalogues and geospatial analysis in Jupyter. Several gaps have been identified in the related work. Specifically, existing work does not support all public catalogues, does not integrate well with the Jupyter ecosystem, and does not streamline the entire scientific workflow, from discovery to extraction.

To address these gaps and to answer the research question, an application was designed. In this application, users can provide a metadata catalogue and browse the metadata records visually on an interactive map. Furthermore, users can then inspect metadata records and preview their data. Finally, users can easily export these datasets to Jupyter to use the data in their analysis. This design was then implemented in the form of a web-based application.

Several usability tests have been conducted on the application by professionals in the geospatial data field. In these usability tests, we have validated several tasks that researchers would perform using the application. The participants of the usability test also took part in an impression test, of which the result was positive. Mostly, participants found the application to be responsive, approachable, accessible, and effective.

The application addresses several growing needs of geospatial data researchers. Namely, the need for effective browsing, selecting and extracting geospatial data from data catalogues. As the amount of public datasets increases, researchers have more options when selecting datasets for their research. It is essential that researchers can effectively make an informed decision on which dataset to use for their research. The application has the potential to accelerate the research process and help researchers select better datasets, leading to more accurate and insightful analysis.

Bibliography

- [1] `@map-colonies/csw-client` - *npm*. May 2023. URL: <https://www.npmjs.com/package/@map-colonies/csw-client>.
- [2] *About OSGeo* - *OSGeo*. June 2023. URL: <https://www.osgeo.org/about/>.
- [3] Felix Atsma. “Interactive time series weather data visualization on Jupyter”. In: (2021).
- [4] Ioana Baldini et al. “Serverless computing: Current trends and open problems”. In: *Research advances in cloud computing* (2017), pp. 1–20.
- [5] Joey Benedek and Trish Miner. “Measuring Desirability: New methods for evaluating desirability in a usability lab setting”. In: *Proceedings of Usability Professionals Association* 2003.8-12 (2002), p. 57.
- [6] David A Bennett and Marc P Armstrong. “Fundamentals of geographic information systems (GIS)”. In: *Manual of Geospatial Science and Technology*, edited by J. Bossler, London: Taylor and Francis (2001), pp. 411–430.
- [7] Peter A Burrough, Rachael A McDonnell, and Christopher D Lloyd. *Principles of geographical information systems*. Oxford university press, 2015.
- [8] Howard Butler et al. *The geojson format*. Tech. rep. 2016.
- [9] *CORS* - *MDN*. May 2023. URL: <https://developer.mozilla.org/en-US/docs/Glossary/CORS>.
- [10] *CSDGM Technical Specification*. Jan. 1998. URL: https://www.fgdc.gov/standards/projects/metadata/base-metadata/v2_0698.pdf.
- [11] `csw-client` - *npm*. May 2023. URL: <https://www.npmjs.com/package/csw-client>.
- [12] *Dublin Core*. May 2023. URL: <https://www.dublincore.org/>.
- [13] *ESRI Shapefile Technical Description*. July 1998. URL: <https://www.esri.com/content/dam/esrisites/sitecore-archive/Files/Pdfs/library/whitepapers/pdfs/shapefile.pdf>.
- [14] *Folium*. URL: <https://python-visualization.github.io/folium/>.
- [15] Open Source Geospatial Foundation. *GeoNetwork opensource*. 2023. URL: <https://geonetwork-opensource.org/>.
- [16] Open Source Geospatial Foundation. *GeoServer*. 2023. URL: <https://geoserver.org/about>.
- [17] *GeoNetwork - Describing Information*. May 2023. URL: <https://geonetwork-opensource.org/manuals/4.0.x/en/user-guide/describing-information/index.html>.
- [18] *GeoNetwork API*. May 2023. URL: <https://sdi.eea.europa.eu/catalogue/doc/api/index.html>.
- [19] Seth Goodman et al. “GeoQuery: Integrating HPC systems and public web-based geospatial data tools”. In: *Computers & geosciences* 122 (2019), pp. 103–112.
- [20] Tao Guo et al. “Efficient selection of geospatial data on maps for interactive and visualized exploration”. In: *Proceedings of the 2018 international conference on management of data*. 2018, pp. 567–582.

- [21] Stefano Iacovella. *GeoServer Beginner's Guide: Share Geospatial Data Using Open Source Standards*. Packt Publishing Ltd, 2017.
- [22] *INSPIRE Metadata Implementing Rules: Technical Guidelines based on EN ISO 19115 and EN ISO 19119*. Nov. 2013. URL: <https://inspire.ec.europa.eu/file/1557/download?token=UaQBcRvQ>.
- [23] *inspire-parser - npm*. May 2023. URL: <https://www.npmjs.com/package/inspire-parser>.
- [24] *ISO 19115*. Apr. 2004. URL: <https://www.iso.org/obp/ui/#iso:std:iso:19115:-1:ed-1:v1:en>.
- [25] *ISO 19139*. Apr. 2007. URL: <https://www.iso.org/obp/ui/#iso:std:iso:ts:19139:-1:ed-1:v1:en>.
- [26] *jsonix - npm*. May 2023. URL: <https://www.npmjs.com/package/jsonix>.
- [27] *Jupyter widgets*. URL: <https://ipywidgets.readthedocs.io/en/latest/?badge=latest>.
- [28] *Leaflet - an open-source JavaScript library for mobile-friendly interactive maps*. May 2023. URL: <https://leafletjs.com/>.
- [29] *Mapbox GL JS*. May 2023. URL: <https://www.mapbox.com/mapbox-gljs>.
- [30] *MapLibre - About*. May 2023. URL: <https://maplibre.org/about/>.
- [31] *MapServer*. May 2023. URL: <https://mapserver.org/>.
- [32] *Nationaal Georegister*. June 2023. URL: <https://www.nationaalgeoregister.nl>.
- [33] *OGC - Standards*. Apr. 2023. URL: <https://www.ogc.org/standards/>.
- [34] *OGC API*. May 2023. URL: <https://ogcapi.ogc.org/>.
- [35] *Open Geospatial Consortium*. Apr. 2023. URL: <https://www.ogc.org/>.
- [36] *OpenGIS Catalogue Services Specification*. Feb. 2007. URL: http://portal.opengeospatial.org/files/?artifact_id=20555.
- [37] *OpenGIS Web Map Service (WMS) Implementation Specification*. Jan. 2006. URL: https://portal.ogc.org/files/?artifact_id=14416.
- [38] *OpenLayers*. May 2023. URL: <https://openlayers.org/>.
- [39] *OpenSearch*. May 2023. URL: <https://opensearch.org/about.html>.
- [40] *OSGeo/ows.js - GitHub*. May 2023. URL: <https://github.com/OSGeo/ows.js>.
- [41] Jeffrey M Perkel. “Why Jupyter is data scientists’ computational notebook of choice”. In: *Nature* 563.7732 (2018), pp. 145–147.
- [42] *pycsw*. May 2023. URL: <https://pycsw.org/>.
- [43] David Reinsel, John Gantz, and John Rydning. “Data age 2025: The evolution of data to life-critical”. In: *Don't Focus on Big Data 2* (2017).
- [44] *SDI - Geospatial Data Catalogue*. URL: <https://sdi.eea.europa.eu/>.
- [45] Alexander K Smith et al. “Conducting high-value secondary dataset analysis: An introductory guide and resources”. In: *Journal of general internal medicine* 26 (2011), pp. 920–929.
- [46] Pierre Soille et al. “A versatile data-intensive computing platform for information retrieval from big geospatial data”. In: *Future Generation Computer Systems* 81 (2018), pp. 30–40.
- [47] *Technical Guidance for the implementation of INSPIRE Download Services*. Aug. 2013. URL: https://inspire.ec.europa.eu/file/1554/download?token=Y_538IH4.
- [48] *The Atom Syndication Format*. Dec. 2005. URL: <https://www.ietf.org/rfc/rfc4287.txt>.
- [49] *The Open Archives Initiative Protocol for Metadata Harvesting*. June 2002. URL: <https://www.openarchives.org/OAI/openarchivesprotocol.html>.

- [50] *What is GeoNode*. May 2023. URL: <https://docs.geonode.org/en/master/about/index.html>.
- [51] *What is raster data? - ArcMap*. URL: <https://desktop.arcgis.com/en/arcmap/latest/manage-data/raster-and-images/what-is-raster-data.htm>.

Appendix

A.1 Source code

The source code of the application designed and developed in this thesis is hosted at <https://github.com/vigovlugt/thesis-app>

A.2 Usability form

On the next pages, the usability form sent to the participants in the usability test is shown.

Usability Test

Thanks for participating in this usability test.

In this usability test, you will test an application developed for my bachelor thesis "Effective Geospatial Data Discovery and Extraction", in which I designed an application that helps researchers to effectively browse, select and extract geospatial data.

The test will be split in 5 tasks, each will test another part of the application.

At the end of the usability test, an impression test will also be performed to measure your impressions regarding the application.

* Indicates required question

Geospatial data recap

Metadata catalogue

Metadata catalogues store information about a collection of datasets. This information is stored in the form of metadata records.

Metadata records

Metadata records contain information about a dataset, this information includes the title, abstract, technical specifications, contact information of the dataset. Importantly, the link to download the actual data is also contained in this information. These metadata records can also reference associated services, which allow the user to preview the data without first downloading the entire dataset.

WFS

WFS ([Web Feature Service](#)) is such a service, this service serves geospatial vector data.

WMS

WMS ([Web Map Service](#)) is another geospatial data service, WMS serves raster data.

CSW

CSW ([Catalogue Service for the Web](#)) is a protocol designed for programmatic interfacing with metadata catalogues. The application you will test is designed around this protocol, allowing you to input the url of any CSW service, and then providing an user interface where you can visually browse this service.

Task 1

Please open the following [link](#) in your browser, which will take you to the entry page of the application.

Here you will see a form which allows you to submit a CSW endpoint.

For this test, we will be using the CSW endpoint of the metadata catalogue of the dutch "Nationaal Georegister" (www.nationaalgeoregister.nl), which stores all kinds of geospatial datasets of the Netherlands.

Please confirm that the [nationaalgeoregister.nl](http://www.nationaalgeoregister.nl) website is up, as it is regularly down and the application can't work if the site is down.

Your first task is:

Open the Nationaal Georegister metadata catalogue in the application.

The CSW endpoint of the catalogue is:

<https://www.nationaalgeoregister.nl/geonetwork/srv/eng/csw>

When you start the task, please start a timer to measure the time it takes to complete the task.

1. Did you successfully complete task 1? *

Mark only one oval.

☐ Yes

☐ No

2. How long did it take to complete task 1?

Metadata record page

You are now in main page of the application.

Here you see a map which displays the bounding boxes of the metadata records of the Nationaal Georegister metadata catalogue.

You also see a table which displays information about these metadata records which are shown on the map.

If you zoom in to the Netherlands and pan around on the map, the records on the map will be updated with records within the current viewpoint.

On the left hand side, you can filter the shown records based on a keyword, or a resource type or topic.

If you click on any record on the map or table, you will see the information within the metadata record on the right hand side of the application.

At the bottom of this panel, you can see the related services, files and links that are stored in the metadata record, these will be important in another task.

Take some time to explore this page, the map, and different datasets. If you are ready, you can continue with the next task.

Task 2

For this task, you will enact a data researcher which is interested in data of historical "molens" in Noord-Holland.

As such, your task is:

Find a dataset about historical molens in Noord-Holland. The record must have a WFS service attached.

When you start the task, please start a timer.

3. Did you successfully complete task 2? *

Mark only one oval.

☐ Yes

☐ No

4. How long did it take to complete task 2?

Task 3

If you succeeded the task, you have found and selected the dataset "Molens uit 1812".

If you did not succeed, reset any filters, enter "Molens 1812" into the search box, then zoom out on the map so the entirety of Noord-Holland is visible and click on the "Molens uit 1812" record in the table or on the map.

Your next task is:

Preview the data in the dataset using the attached WFS service.

When you start the task, please start a timer.

5. Did you successfully complete task 3? *

Mark only one oval.

☐ Yes

☐ No

6. How long did it take to complete task 3?

Service page

You should now find yourself on the page where you can see the dataset on the map.

If you did not succeed, open the following [link](#), where you will be routed to the correct page.

On the right side, you can still see the metadata record information, left to that, you can see the WFS service information.

The map should now show the features stored in the WFS service; the mills from 1812.

You can click a mill to view its metadata, you can also select multiple mills holding ctrl or shift.

By dragging a box, you are able to select a region of mills.

Task 4

An important aspect of the application is its integration with Jupyter.

Please create a Jupyter notebook or easily create a new Google Colab notebook using the following link:

<https://colab.research.google.com/#create=true>.

Your task is:

Import the "Molens in 1812" dataset in Jupyter.

When you start the task, please start a timer.

7. Did you successfully complete task 4? *

Mark only one oval.

☐ Yes

☐ No

8. How long did it take to complete task 4?

Task 5

In the final task, the usability of the subset selection and extraction feature will be tested.

Your task is:

Select and import 5 features (in this case mills) of the "Molens in 1812" dataset in Jupyter.

When you start the task, please start a timer.

9. Did you successfully complete task 5? *

Mark only one oval.

☐ Yes

☐ No

10. How long did it take to complete task 5?

Impression test

Finally, to measure your impression of the application, I will ask you to circle 5 words you associate most with the application.

11. Select 5 of the following 25 impressions that you think identify the application

Check all that apply.

- ☐ Accessible
- ☐ Advanced
- ☐ Annoying
- ☐ Approachable
- ☐ Attractive
- ☐ Boring
- ☐ Complex
- ☐ Comprehensive
- ☐ Confusing
- ☐ Easy to use
- ☐ Effective
- ☐ Empowering
- ☐ Exciting
- ☐ Friendly
- ☐ Hard to Use
- ☐ Inconsistent
- ☐ Ineffective
- ☐ Intuitive
- ☐ Novel
- ☐ Responsive
- ☐ Time-consuming
- ☐ Time-saving
- ☐ Unattractive
- ☐ Unpredictable
- ☐ Valuable

12. Do you have any other impressions, suggestions, or ideas about the application?

Thank you!

Thank you for participating in this study!

Hopefully you found the application interesting and thank you for your time. Your contribution is very valuable for my thesis.

This content is neither created nor endorsed by Google.

Google Forms

